# *ACTIVE MIND EXPLORER – APP PRESENTATION*

# (AME)

**Bălălău Andrei-Valentin**

January 2026

# 1. Introduction - Overview, main ideas and architecture

## 1.1 Introduction, theme and functionality

Active Mind Explorer (AME) is a distributed system designed as a client-server architecture, consisting of two independent sub-applications communicating over a network.

The project's theme is tracking online course attendance for an educational platform. It combines web application mechanisms (Java backend) with database management into a single package, performing CRUD operations across seven SQL Server tables.

The architecture strictly separates the graphical user interface (client-side) from the data processing logic (server-side), ensuring better scalability, easier maintenance, and enhanced security. The database structure and functionalities were inspired by a 6-week OSF Academy training program focused on Salesforce cloud databases.

## 1.2 Database definition and core features

The database is designed to handle multiple complex relationships, including many-to-many and one-to-many associations. Here is the breakdown of the 7 main tables and their roles:

- ***Users (Utilizatori):***
    - **Role:** Stores all participants, including students, trainers, and platform administrators.
    - **Key Fields:** UserID (Primary Key), TrainerID (Foreign Key for self-referencing), FirstName, LastName, Email, Password, RoleType, NumberOfBadges.
    - **Constraints:** The Email must be UNIQUE, and the RoleType is restricted to 'Cursant' (Student), 'Trainer' or 'Colab' (Collaborator).
- ***Courses (Cursuri):***
    - **Role:** Contains details about the training programs (e.g., "Salesforce Admin").
    - **Key Fields:** CourseID (Primary Key), Name, Description, CourseDuration.
- ***Enrolments (Înscrieri):***

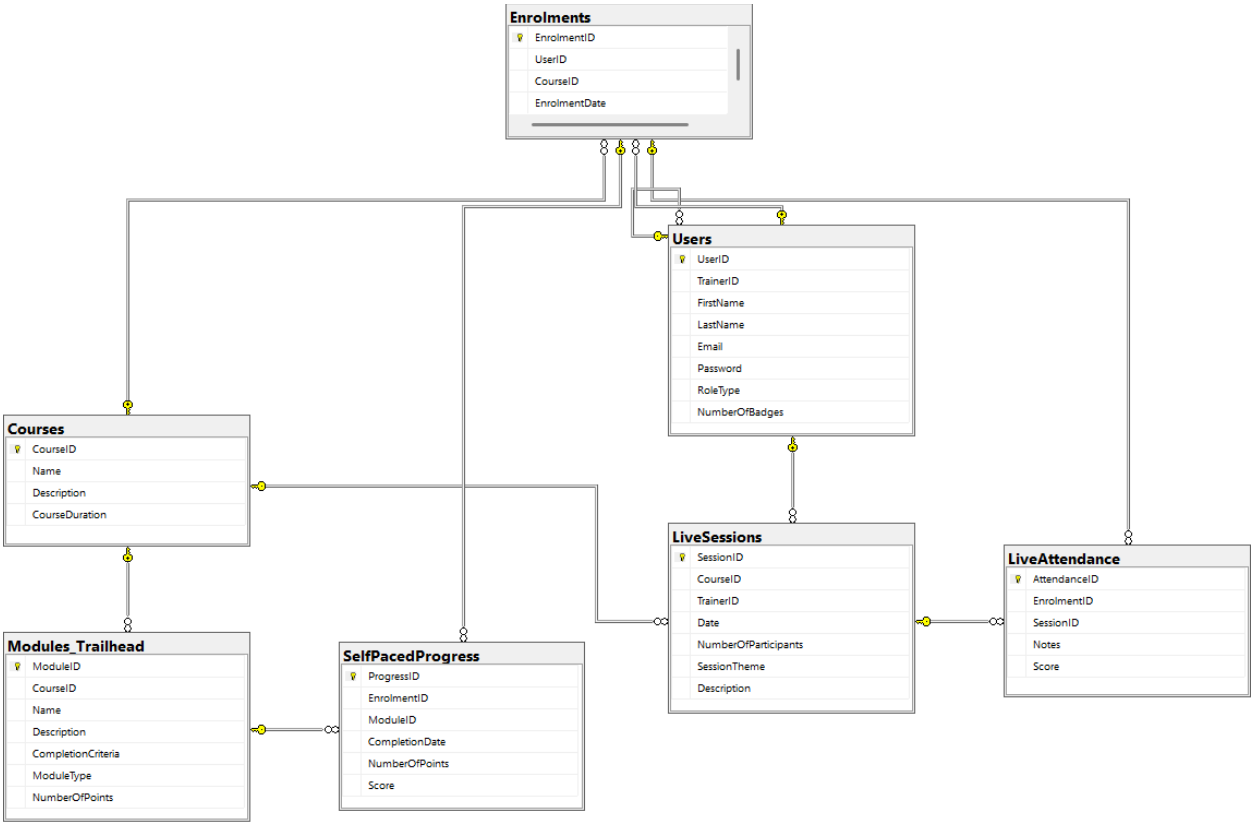
- **Role:** Acts as a junction table to resolve the many-to-many (N:N) relationship between Users and Courses (because a user can take multiple courses and a course has multiple users).
- **Key Fields:** EnrolmentID (Primary Key), UserID, CourseID, EnrolmentDate.

- ***Modules_Trailhead:***
  - **Role:** Represents the self-paced learning components (lessons, projects, badges);It has a one-to-many (1:N) relationship with Courses (one course contains multiple modules).
  - **Key Fields:** ModuleID (Primary Key), CourseID, Name, Description, CompletionCriteria, ModuleType, NumberOfPoints.
- ***SelfPacedProgress (Progres_Asincron):***
  - **Role:** Tracks the completion of Trailhead modules and the badges obtained by each student.
  - **Key Fields:** ProgressID (Primary Key), EnrolmentID, ModuleID, CompletionDate, NumberOfPoints, Score.
- ***LiveSessions (Sesiune_Live):***
  - **Role:** Stores details about the weekly online discussion meetings with the trainers.
  - **Key Fields:** SessionID (Primary Key), CourseID, TrainerID, Date, NumberOfParticipants, SessionTheme, Description.
- ***LiveAttendance (Participare_Live):***
  - **Role:** Tracks the active presence and engagement (score) of each student at a specific live session. It resolves the N:N relationship between Enrolments and LiveSessions.
  - **Key Fields:** AttendanceID (Primary Key), EnrolmentID, SessionID, Notes, Score.

## 1.3 The Visual Representation (Database Diagram Overview)

Based on the Entity-Relationship Diagram (ERD) provided bellow, the architecture connects these tables logically:

- The **Enrolments** table sits at the center of the student's learning journey, linking the Users directly to the Courses.
- From the Enrolments table, a student's progress branches out into two distinct paths:
    1. **The Asynchronous Path:** Links to SelfPacedProgress, which in turn connects to Modules_Trailhead (the specific lessons tied to a Course).
    2. **The Synchronous Path:** Links to LiveAttendance which connects to the LiveSessions scheduled for that specific Course.
- The **Users** table features a recursive (self-referencing) relationship. This means a trainer is stored in the exact same table as a student, and a student's row simply points to their assigned trainer's ID via the TrainerID field.



## 2. Technical Choices and Data Structures

- ***Date Management:*** LocalDate (from java.time) was chosen over the legacy java.util.Date for accurate, timezone-independent calendar date management.
- ***Integer vs. int:*** The wrapper class Integer is used for database IDs instead of primitive int to properly handle NULL values (e.g., when a student is not yet assigned a trainer).
- ***Singleton pattern:*** A UserSession class implements the Singleton pattern to maintain the logged-in user's state globally across the client application.
- ***MVC architecture:*** Both layers follow the Model-View-Controller pattern. The JavaFX client handles UI, while the Spring Boot server manages business logic via Services and Repositories.

## 3. Backend implementation (Spring Boot & MVC)

The backend was generated using Spring Initializr, incorporating Spring Web, Spring Data JPA, and the SQL Server Driver. Complex operations (such as deleting a user) are managed via *Service* layers to ensure data integrity. For example, a cascading delete manually removes self-paced progress, live attendance and enrolments before finally deleting the user record.

## 4. SQL Functionalities and Queries

To bypass standard ORM limitations and optimize performance, native SQL queries (@Query) are heavily utilized:

- ***CRUD operations:*** Standard insert, update and delete statements for database entities.
- ***Aggregate functions:*** Basic queries utilizing COUNT, AVG and SUM to calculate metrics like course averages and total scores resulted from live sessions / badge progress.
- ***JOINs:*** Simple and multiple JOINs to correlate data, such as retrieving the list of users enrolled in a specific course.
- ***Complex subqueries:*** Filtering advanced scenarios using EXISTS, NOT IN and scalar subqueries (e.g – keeping track of the enrolled students/ trainees).

- **Parameterized Queries:** All SQL statements use parameters (@Param) to prevent SQL injection and allow dynamic real-time tracking.

## 5. Communication, Optimization and Validation

- **DAO Pattern -** The client relies on Data Access Objects (DAO) using *HttpClient* to communicate seamlessly with the server's REST API, hiding network complexities from the UI controllers.
- **Jackson -** Used to serialize/deserialize Java objects to and from JSON format.
- **Server-side calculations -** Computations (like averages) are pushed to the database via native SQL to minimize network payload and improve UI efficiency.
- **Dynamic UI-** The interface automatically adapts to user roles, displaying specific administrative buttons only for authorized personnel/ staff members.

## 6. App Functionalities (User Experience)

- **Login Screen -** Secure entry point providing real-time feedback and hidden password fields.
- **Dashboard -** A central command interface that visually adapts to the logged-in user's role and time of day.
- **User & Course management panels/tools -** Feature custom ListViews, real-time detail panels and parallel thread execution (Task<Map>) to prevent the UI from freezing during database fetches.
- ***Gamification* -** Automatic badge calculation logic (e.g., scoring >50% on a "SuperBadge" module yields 3 badges).
- **Dark Mode -** A fully integrated dark theme managed purely via *JavaFX CSS*, persisting across all screens through the UserSession Singleton.

## 7. Potential future Improvements

Proposed enhancements for expanding the application's functionality include:

- A global badge leaderboard and experience level system

- Daily streak visual notifications to increase student engagement
- Integration of an email service (*JavaMailSender*) for automated notifications (e.g., grade updates or upcoming live sessions)
- A dedicated "Analytics" dashboard using *JavaFX Charts* for advanced visual data reporting